prfaq

A virtual product manager for engineers building with AI coding tools

February 2026 | punt-labs

Press Release

Summary

SAN FRANCISCO — Today, punt-labs released `prfaq`, a free, open-source Claude Code plugin that guides engineers and solo founders through Amazon's Working Backwards PR/FAQ process to evaluate product ideas before writing code. Unlike blog posts, templates, or workshops that sit outside the developer's workflow, `prfaq` runs as a slash command inside the same Claude Code session where the engineer already builds software, producing a professionally typeset PDF in under an hour. Engineers who ship features at unprecedented speed can now apply the same rigor to deciding *which* features to ship.

Problem

AI coding tools have made it possible for a single engineer to build in a weekend what used to take a team a quarter. The bottleneck has shifted: the hard part is no longer writing code — it is knowing what to build and why (see p. 4). Engineers and solo founders using Claude Code, Cursor, and similar tools ship fast, but most lack formal product training. They skip customer definition, skip competitive analysis, skip unit economics — and discover after burning through tokens and weeks of building that the product does not resonate.

The consequences are visible. Analysts at HFS Research estimate the Global 2000 carry $1.5–2 trillion in accumulated technical debt [1], and AI-generated code is adding to the pile faster than ever. Thousands of startups built during the current vibe coding wave already face significant rebuilds — not because the code is bad, but because the code solves the wrong problem with confidence.

Today, an engineer who wants product discipline has three options: hire a product manager they cannot afford, read a book and try to apply frameworks manually, or just iterate — ship something, get feedback, ship again. Iteration works. Some of the best developer tools started as something an engineer built for themselves, and others appreciated. But iteration has real costs: every cycle burns tokens, burns calendar time marketing to early users, and burns the founder's energy. Getting customer feedback is often the actual bottleneck. Plenty of engineers would develop stronger product sense if the frameworks were more accessible — available in the workflow where they already build, not in a book or a workshop they will never attend.

Solution

`prfaq` brings product thinking directly into the engineer's development environment. By typing `/prfaq` in any Claude Code session, the engineer starts a structured conversation: Claude

asks about the target customer, the problem, the competitive landscape, and the business model. It asks hard questions — the kind a product manager would ask before greenlighting a project.

From these answers, Claude generates a complete PR/FAQ document: a one-page mock press release describing the product as if it has already shipped, followed by external FAQs (what a customer would ask) and internal FAQs (what leadership, finance, and engineering would ask). The document is evaluated against Marty Cagan's four risks framework [2] — value, usability, feasibility, and viability — producing an honest assessment of whether the product is worth building.

The output is a LaTeX-compiled PDF (see p. 3): a clean, professional artifact suitable for sharing with co-founders, investors, or advisors. It is not a disposable brainstorm. It is a decision-making document designed to be read, debated, and revised.

Customer Quote

> *"I kept building MVPs that nobody used. I'd spend three weeks coding, show it to people, and get polite shrugs.* `prfaq` *made me realize I was solving my own problem, not my customer's. The PR/FAQ took forty-five minutes and saved me a month of wasted work. The hardest question it asked was 'what evidence do you have that customers want this?' I didn't have any. That was the answer."*
>
> — **Marcus Okonkwo**, Solo Founder, SyncPilot

Getting Started

Getting started with `prfaq` takes three steps. First, run the one-line installer: `curl -fsSL https://punt-labs.github.io/prfaq/install | bash`. This clones the plugin repository and registers it with Claude Code automatically. Second, open any Claude Code session and type `/prfaq`. Third, answer the discovery questions — Claude guides the conversation, asks follow-ups, and drafts the PR/FAQ document. Within an hour, the engineer has a compiled PDF that forces clarity on customer, problem, differentiation, and business model. No account required, no subscription, no configuration.

Spokesperson Quote

> "Engineers are the best product thinkers in the room — they just don't always have the frameworks. We built `prfaq` because the engineers using Claude Code every day are also the ones making product decisions (see p. 4), and they deserve a tool that meets them where they already work. The PR/FAQ process has been proven at Amazon for two decades [3]. We didn't invent it. We made it accessible inside a terminal."
>
> — **punt-labs**

Call to Action

`prfaq` is free and open source, available now at `https://github.com/punt-labs/prfaq`. Install it, type `/prfaq`, and find out if your next product idea is worth building before you build it.

Frequently Asked Questions

External FAQs

Q: What is prfaq and who is it for?

`prfaq` is a free Claude Code plugin that guides engineers and solo founders through the Amazon Working Backwards PR/FAQ process. It is for anyone who builds software with AI coding tools and wants to validate a product idea before committing weeks or months of development time. If you make product decisions — what to build, for whom, and why — this tool is for you.

Q: How is this different from using a PR/FAQ template?

A template is a blank page with headings. `prfaq` is an interactive process: Claude asks discovery questions, challenges weak answers, identifies gaps in customer evidence, and generates the document from your responses. It applies the four risks framework to evaluate your idea honestly, not just format it. A template tells you *what* to write. `prfaq` helps you figure out *what to think*.

Q: How do I get started?

Run the one-line installer (`curl -fsSL https://punt-labs.github.io/prfaq/install | bash`), then type `/prfaq` in any Claude Code session. The plugin walks you through the entire process. No account, no API keys, no configuration. Installation takes under a minute.

Q: What does the output look like?

A professionally typeset PDF compiled from LaTeX. The document includes a one-page press release, external and internal FAQs, and a four-risks assessment. It is designed to be shared with co-founders, investors, or advisors — not a throwaway brainstorm document.

Q: Do I need to know LaTeX?

No. The plugin generates the LaTeX source and compiles it automatically. You never see or edit LaTeX unless you choose to. You need a TeX distribution installed (TeX Live or similar), which the installer checks for and provides setup instructions if missing.

Q: Why LaTeX instead of Markdown?

Three reasons. First, a PR/FAQ is a decision document — it gets shared with co-founders, investors, and advisors. LaTeX produces professionally typeset PDFs with consistent typography, precise layout, and visual credibility that Markdown-to-PDF pipelines do not match. The medium signals that the thinking is serious. Second, LaTeX's structured environments (`faqpair`, `riskitem`, `customerquote`) give the LLM a precise grammar to write into — the AI reads and writes `.tex`, the human reads the compiled `.pdf`. This separation means the source format can be rich and semantic without burdening the author. Third, LaTeX's package ecosystem (biblatex for citations, mdframed for callout boxes, enumitem for structured lists) provides extensibility that Markdown lacks without custom renderers.

The trade-off is friction: LaTeX requires a TeX distribution (~4 GB), and users unfamiliar with it may find compilation errors intimidating. The installer mitigates this with automated setup, and the plugin handles all LaTeX generation — the user never writes `.tex` directly. This is a two-way

door: if user feedback shows the TeX dependency is a meaningful adoption barrier, adding a Markdown output mode is straightforward since the content generation is format-independent. The structured thinking matters more than the output format.

Q: What happens to my data?

Everything runs locally. The plugin is a set of prompt files and a LaTeX template — it does not phone home, collect telemetry, or store your data anywhere. Your PR/FAQ content is processed by Claude Code using your existing Anthropic API relationship. The generated `.tex` file and compiled PDF stay on your machine. Data handling is governed entirely by Claude Code's existing privacy model and your Anthropic terms of service.

Q: How much does it cost?

Free. `prfaq` is open source under a permissive license. There is no paid tier, no freemium upsell, and no planned monetization. You do need an active Claude Code subscription or API access, which is a separate cost from Anthropic. The plugin itself adds zero cost.

Q: Can I iterate on the document?

Yes. The plugin generates a `.tex` file in your project directory. You can re-run the process, edit sections, or ask Claude to revise specific parts. The Working Backwards process is designed for rapid iteration — the point is to iterate on a one-page document instead of iterating on a shipped product.

Internal FAQs

Value & Market

Q: What is the total addressable market?

The primary market is engineers and solo founders using AI coding tools to build products. As of early 2026, over 80% of professional developers use AI coding assistants regularly (daily or weekly) [4]. Claude Code has surpassed $2B in annualized revenue [5] and is among the fastest-growing AI coding tools. The subset of these users who are making product decisions — solo founders, indie hackers, startup engineers, and technical co-founders — numbers in the hundreds of thousands. This is not a TAM-based revenue argument (the plugin is free); it is a reach argument. A useful, free plugin in a fast-growing ecosystem reaches users through organic discovery.

Q: What evidence do we have that customers want this?

Three categories of indirect evidence, but no primary customer data yet. First, the "vibe coding" phenomenon: thousands of products are being built at speed without product discipline, and rebuilds are already visible across startup communities and engineering forums. Second, the explosion of "how to think about product as an engineer" content — books, courses, and newsletters targeting technical founders suggest unmet demand for product skills in developer contexts. Third, the Working Backwards process itself has two decades of validation at Amazon [3], where it is mandatory for new product proposals. The gap is not whether the framework works — it is accessibility. Engineers do not attend PM workshops. They install plugins. *Note:*

we have not yet validated that engineers want product frameworks delivered inside their terminal. Five to ten customer discovery interviews would significantly strengthen this evidence base.

Q: Who are the competitors and why will we win?

The nearest competitor is not a product management tool — it is the user pasting a PR/FAQ template into Claude.ai or any AI chat and having a free-form conversation. This works for experienced practitioners who already know the framework. What it lacks: structured reference guides that catch common mistakes, the four risks evaluation framework, automated peer review against cognitive biases, LaTeX compilation to a shareable PDF, and a repeatable process that improves across uses. The plugin encodes twenty years of Working Backwards practice into a reusable workflow; a blank chat starts from zero every time.

Beyond that, traditional competitors include product management tools (Productboard, Aha!, Notion templates) and business planning tools (Lean Canvas generators). None of these are integrated into a developer's coding environment. They all require context-switching: leave the terminal, open a browser, fill in a form. A Claude Code plugin runs in the same session where the engineer builds software — the mental model shift from "coding" to "product thinking" happens in a single command, not a tab switch.

Technical

Q: What are the major technical risks?

The technical risks are minimal. The plugin is a set of markdown files (skill definition and reference guides) plus a LaTeX template and a shell script. It requires no backend, no API, no database, no infrastructure. The primary technical dependency is the Claude Code plugin system, which is stable and well-documented. The LaTeX dependency requires users to have a TeX distribution installed, which is a mild friction point for users who do not already have one. Mitigation: the installer checks for TeX and provides platform-specific setup instructions.

Q: What dependencies exist on other teams or systems?

The plugin depends on Anthropic's Claude Code plugin system for loading and execution, and on a local TeX distribution for PDF compilation. Both are external dependencies outside our control. The Claude Code plugin API is stable (v1), and TeX Live is a mature, 30-year-old ecosystem. If Anthropic changes the plugin format, the migration would be straightforward — the plugin is a handful of files with simple structure. If TeX Live is unavailable, the plugin still generates the `.tex` source file; only PDF compilation is affected.

Q: If this succeeds, what breaks first?

Nothing at the infrastructure level — the plugin is stateless, runs locally, and requires no servers. At scale, the pressures are human, not technical: (1) GitHub issue volume — at 1,000+ users, support requests and feature demands will exceed solo maintainer capacity; mitigation is clear contribution guidelines and aggressive scope discipline (see Won't Do list); (2) prompt maintenance — as Claude models evolve, skill prompts may need tuning to maintain output quality; (3) backwards compatibility — if Anthropic changes the plugin system, the plugin must migrate; the simple file-based architecture makes this low-risk; (4) community expectations — a popular free tool attracts requests for features that contradict the design philosophy (dashboards,

collaboration, other LLM support). The Won't Do list exists to make these decisions in advance, not under pressure.

Q: What is the estimated development timeline?

The plugin is built. Initial development took one session. Ongoing work is iterative: improving the skill prompts based on real usage, expanding reference material, and potentially adding alternative output formats. There is no multi-month roadmap because there is no infrastructure to build.

Business

Q: What is the revenue model?

There is none. `prfaq` is free and open source under a permissive license. The strategic value is threefold: (1) it is a proving ground for the Claude Code plugin development pattern, informing future commercial plugins; (2) it builds reputation and visibility in the Claude Code ecosystem; (3) it creates a feedback loop for understanding how engineers approach product thinking, which informs other products at punt-labs.

Q: What are the key metrics for success?

Three metrics, measured via GitHub analytics and community signals:

1. **GitHub stars and forks** — target: 500 stars within 6 months. This measures discoverability and perceived value.
2. **Issues and pull requests from external contributors** — target: 10 external PRs within 6 months. This measures whether the plugin is useful enough that people invest time improving it.
3. **Mentions in developer communities** (Hacker News, Twitter/X, Reddit, Discord servers) — target: 3 organic mentions within 3 months. This measures whether the plugin solves a real problem that people tell others about.

If none of these targets are met after 6 months, the plugin is not reaching its audience and the distribution strategy should be reconsidered.

Q: Why now? What has changed?

Three shifts converged. First, AI coding tools reached mainstream adoption in 2025–2026, creating a large population of engineers who can build products independently but lack product discipline. Second, Claude Code's plugin system matured, making it possible to deliver structured workflows inside the developer's existing tool. Third, the consequences of "vibe coding without product thinking" are becoming visible — failed launches, rebuilds, wasted months — creating demand for lightweight product frameworks that do not require hiring a PM or leaving the terminal.

Q: What is the customer acquisition strategy?

Distribution is the product strategy for a free plugin. Primary channels: (1) GitHub discoverability — README, topics, and stars drive organic search traffic; (2) developer community seeding — posts on Hacker News, Reddit r/ClaudeAI, and Twitter/X targeting the "vibe coding" audience; (3) content marketing — the plugin's own PR/FAQ serves as a case study and

demo artifact; (4) word-of-mouth — engineers who find the plugin useful share the PDF output, which includes a link back to the repository. There is no paid acquisition. If organic channels do not work within 6 months, the distribution hypothesis is wrong and should be revisited.

Q: What are we not building?

We are not building a SaaS product management platform. We are not building a competitor to Jira, Linear, Productboard, or Notion. We are not adding collaboration features, dashboards, or analytics. We are not building a web interface. The plugin is a single-player tool that runs locally, produces a document, and gets out of the way. Scope discipline is the feature. See the Feature Appendix for the full Must Do / Should Do / Won't Do breakdown.

Q: It is one year from now and prfaq failed. What went wrong?

Most likely failure scenario: engineers do not want product discipline, even when it is free and frictionless. The vibe coding audience builds for the joy of building, and a tool that asks "what evidence do you have that customers want this?" feels like a buzzkill, not a feature. Second scenario: the Claude Code plugin ecosystem does not grow — if Anthropic deprioritizes plugins or the marketplace never materializes, distribution collapses. Third scenario: the output quality is not high enough to share with stakeholders — if the generated PR/FAQ reads like AI slop rather than a credible decision document, users will not trust it for real decisions. Fourth scenario: the PR/FAQ process is too heavyweight for the target audience — engineers want a quick sanity check, not a formal document, and a lighter-weight alternative captures the market.

Risk Assessment

Four Risks Assessment

Value — Medium

Engineers and solo founders demonstrably lack product discipline, and the consequences (wasted builds, failed launches) are real and growing. However, the evidence is market-level, not customer-level — we have not yet interviewed individual users who say "I would use a PR/FAQ plugin in my terminal." The demand for the *framework* is proven (Amazon, two decades). The demand for *this delivery mechanism* is hypothesized. Risk mitigation: the plugin is free, so the adoption barrier is low — users risk minutes, not money.

Usability — Low

One-line install, one slash command to start. Claude guides the entire process interactively. No configuration, no account creation. The LaTeX dependency is the main friction point; the installer checks for it. Time to value is under one hour.

Feasibility — Low

The plugin is built and functional. No backend, no infrastructure, no ongoing operational cost. The hard engineering problem (interactive AI guidance) is solved by Claude itself. The plugin is structured prompts and a document template.

Viability — Low

No revenue target, no operational cost, no support burden. Strategic asset for learning plugin development patterns and building ecosystem presence. The only viability risk is opportunity cost, which is minimal given the small codebase (under 20 files, no dependencies beyond TeX).

Feature Appendix

Must Do (V1)

- Interactive discovery workflow — structured questions that guide the user from idea to document
- Complete PR/FAQ document generation — press release, external FAQs, internal FAQs, four risks assessment
- LaTeX compilation to professional PDF — shareable artifact, not a disposable brainstorm
- Reference guides — encoded best practices for PR structure, FAQ structure, four risks, and common mistakes
- Revise mode — surgical edits to an existing document without regenerating from scratch
- Peer review agent — automated critique against Working Backwards principles and cognitive bias checklist

Should Do (Future Iterations)

- Research sub-agent — scan local `./research/` directory and optional external data sources for evidence
- Biblatex citation system — academic-style citations for claims, linking evidence to assertions
- Unit economics reference guide — structured framework for products with revenue models
- Principal engineer reference guide — deeper feasibility and technical risk evaluation
- UX bar raiser reference guide — usability risk ownership and evaluation
- Judgment-to-FAQ cross-referencing — link flagged judgment calls to FAQ answers that explain reasoning

Won't Do

- **Web dashboard or SaaS platform** — the plugin runs locally inside Claude Code. Adding a web layer would introduce infrastructure, authentication, and hosting costs that contradict the zero-ops design. If users want to share documents, they share the PDF.
- **Collaboration or multi-user editing** — PR/FAQ is a single-author process by design. The document is written by one person, then reviewed by others. Real-time collaboration would encourage design-by-committee, which the Working Backwards process explicitly avoids.
- **Project management features** — no task tracking, no roadmap views, no sprint planning. The plugin produces a decision document, not a project plan. Users who need project management have Linear, Jira, or beads.
- **Alternative LLM backends** — the plugin is a Claude Code plugin. Supporting OpenAI, Gemini, or local models would fragment the prompt engineering, dilute quality, and multiply the testing surface for no clear user benefit.

References

[1] HFS Research. *The Technology Debt Crisis: Quantifying the Hidden Cost of Legacy Systems*. Market research estimating Global 2000 accumulated technical debt at $1.5–2 trillion. 2024.

[2] Marty Cagan. *Inspired: How to Create Tech Products Customers Love*. 2nd ed. Wiley, 2018. ISBN: 978-1119387503.

[3] Colin Bryar and Bill Carr. *Working Backwards: Insights, Stories, and Secrets from Inside Amazon*. St. Martin's Press, 2021. ISBN: 978-1250267597.

[4] Stack Overflow. *2025 Developer Survey*. Annual survey of 65,000+ developers on tools, practices, and AI adoption. 2025. URL: https://survey.stackoverflow.co/2025.

[5] The Information. *Anthropic Revenue Reaches $2.5B Annualized Run Rate*. Financial reporting on Claude Code and API revenue growth. Feb. 2026.